BÀI THỰC HÀNH TEMPLATE METHOD PATTERN

✅ Backend: ASP.NET Core Web API
✅ Frontend: React (gọi API, chọn phương thức thanh toán)
✅ Áp dụng Template Method đúng chuẩn GoF
✅ Có quy trình xử lý thanh toán cố định + cho phép override từng bước

---

💮 BÀI TOÁN

Hệ thống E-commerce có nhiều phương thức thanh toán, nhưng **quy trình xử lý luôn giống nhau**:

1. Validate dữ liệu
2. Tạo transaction
3. Thực hiện thanh toán
4. Ghi log / trả kết quả

👉 Tuy nhiên:

- PayPal xử lý khác
- VNPay xử lý khác

→ Cần dùng **Template Method** để:

- Giữ **flow chung cố định**
- Cho phép các subclass **override từng bước**

📦 KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
        ↓
POST /api/orders/checkout
        ↓
OrderService
        ↓
PaymentProcessor (Template Method)
        ↓
PaypalProcessor | VNPayProcessor
```

💻 BACKEND – ASP.NET Core 8 Web API

1️⃣ Tạo project

```
dotnet new webapi -n TemplatePaymentDemo
cd TemplatePaymentDemo
```

🗂 Cấu trúc thư mục

```
TemplatePaymentDemo
 ├── Controllers
 │      └── OrdersController.cs
 ├── Services
 │      ├── OrderService.cs
 │      ├── PaymentProcessors
 │             ├── PaymentProcessor.cs   (Template)
 │             ├── PaypalProcessor.cs
 │             └── VNPayProcessor.cs
 ├── Models
 │      ├── CheckoutRequest.cs
 │      └── PaymentResult.cs
 └── Program.cs
```

💮 1. MODEL

Models/PaymentResult.cs

```
namespace TemplatePaymentDemo.Models;

public class PaymentResult
{
    public bool Success { get; set; }
    public string Message { get; set; }
}
```

Models/CheckoutRequest.cs

```
namespace TemplatePaymentDemo.Models;

public class CheckoutRequest
```

```
{
    public string PaymentMethod { get; set; }
    public decimal Amount { get; set; }
}
```

---

2. TEMPLATE METHOD (QUAN TRỌNG NHẤT)

Services/PaymentProcessors/PaymentProcessor.cs

```
using TemplatePaymentDemo.Models;

namespace TemplatePaymentDemo.Services.PaymentProcessors;

public abstract class PaymentProcessor
{
    // TEMPLATE METHOD (không cho override)
    public async Task<PaymentResult> Process(decimal
amount)
    {
        Validate(amount);

        var transactionId = await
CreateTransaction(amount);

        var result = await ExecutePayment(amount);

        await Log(transactionId, result);

        return result;
    }

    // Các bước có thể override
    protected virtual void Validate(decimal amount)
    {
        if (amount <= 0)
            throw new Exception("Invalid amount");
    }

    protected virtual async Task<string>
CreateTransaction(decimal amount)
    {
        await Task.Delay(200);
```

```
        return Guid.NewGuid().ToString();
    }

    protected abstract Task<PaymentResult>
ExecutePayment(decimal amount);

    protected virtual async Task Log(string transactionId,
PaymentResult result)
    {
        await Task.Delay(100);
        Console.WriteLine($"Transaction {transactionId}:
{result.Message}");
    }
}
```

👉 Đây chính là **Template Method Pattern chuẩn GoF**

- `Process()` = template method (fixed flow)
- Các method khác = hook / abstract

💮 3. CONCRETE IMPLEMENTATIONS

PaypalProcessor.cs

```
using TemplatePaymentDemo.Models;

namespace TemplatePaymentDemo.Services.PaymentProcessors;

public class PaypalProcessor : PaymentProcessor
{
    protected override async Task<PaymentResult>
ExecutePayment(decimal amount)
    {
        await Task.Delay(500);

        return new PaymentResult
        {
            Success = true,
            Message = $"Paid {amount} via PayPal"
        };
    }
```

```
}
```

VNPayProcessor.cs

```
using TemplatePaymentDemo.Models;

namespace TemplatePaymentDemo.Services.PaymentProcessors;

public class VNPayProcessor : PaymentProcessor
{
    protected override async Task<PaymentResult>
ExecutePayment(decimal amount)
    {
        await Task.Delay(500);

        return new PaymentResult
        {
            Success = true,
            Message = $"Paid {amount} via VNPay"
        };
    }

    // Override thêm bước Validate riêng
    protected override void Validate(decimal amount)
    {
        base.Validate(amount);

        if (amount < 10000)
            throw new Exception("VNPay requires minimum
10,000 VND");
    }
}
```

💮 4. ORDER SERVICE (CHỌN PROCESSOR)

Services/OrderService.cs

```
using TemplatePaymentDemo.Models;
using TemplatePaymentDemo.Services.PaymentProcessors;

namespace TemplatePaymentDemo.Services;
```

```
public class OrderService
{
    private readonly IServiceProvider _serviceProvider;

    public OrderService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public async Task<PaymentResult> Checkout(string
method, decimal amount)
    {
        PaymentProcessor processor = method.ToLower()
switch
        {
            "paypal" =>
_serviceProvider.GetRequiredService<PaypalProcessor>(),
            "vnpay" =>
_serviceProvider.GetRequiredService<VNPayProcessor>(),
            _ => throw new Exception("Unsupported payment
method")
        };

        return await processor.Process(amount);
    }
}
```

👉 Khác Factory:

- Không tạo object logic phức tạp
- Chỉ chọn implementation → focus vào **workflow**

💮 5. CONTROLLER

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using TemplatePaymentDemo.Models;
using TemplatePaymentDemo.Services;

namespace TemplatePaymentDemo.Controllers;
```

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrdersController(OrderService orderService)
    {
        _orderService = orderService;
    }

    [HttpPost("checkout")]
    public async Task<IActionResult>
Checkout(CheckoutRequest request)
    {
        var result = await _orderService.Checkout(
            request.PaymentMethod,
            request.Amount);

        return Ok(result);
    }
}
```

---

6. ĐĂNG KÝ DI

Program.cs

```
using TemplatePaymentDemo.Services;
using TemplatePaymentDemo.Services.PaymentProcessors;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddTransient<PaypalProcessor>();
builder.Services.AddTransient<VNPayProcessor>();
builder.Services.AddTransient<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                  .AllowAnyHeader()
                  .AllowAnyMethod();
        });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseCors("AllowReact");

app.MapControllers();
app.Run();
```

▶ CHẠY BACKEND

```
dotnet run
```

Swagger:

```
https://localhost:xxxx/swagger
```

Test API:

POST `/api/orders/checkout`

```
{
  "paymentMethod": "paypal",
  "amount": 100
}
```

💻 FRONTEND – REACT

Tạo project

```
npx create-react-app template-ui
cd template-ui
npm install axios
```

---

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [method, setMethod] = useState("paypal");
  const [amount, setAmount] = useState(100);
  const [result, setResult] = useState("");

  const checkout = async () => {
    const response = await axios.post(
      "https://localhost:xxxx/api/orders/checkout",
      {
        paymentMethod: method,
        amount: amount,
      }
    );

    setResult(response.data.message);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Template Method Checkout</h2>

      <label>Amount:</label>
      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
      />

      <br /><br />
```

```
      <label>Payment Method:</label>
      <select value={method} onChange={(e) =>
setMethod(e.target.value)}>
        <option value="paypal">PayPal</option>
        <option value="vnpay">VNPay</option>
      </select>

      <br /><br />

      <button onClick={checkout}>Pay Now</button>

      <h3>{result}</h3>
    </div>
  );
}

export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

🔥 LUỒNG XỬ LÝ TEMPLATE METHOD

Khi user chọn PayPal

1. React gửi request
2. OrderService chọn `PaypalProcessor`
3. Gọi:

```
processor.Process(amount);
```

4. Flow chạy:

```
Validate()
→ CreateTransaction()
→ ExecutePayment() (override bởi Paypal)
→ Log()
```

Khi chọn VNPay

Flow giống hệt, nhưng:

- `Validate()` bị override
- `ExecutePayment()` khác

🎯 SO SÁNH NHANH (RẤT QUAN TRỌNG CHO SINH VIÊN)

Pattern	Mục đích
Factory Method	Tạo object
Template Method	Định nghĩa quy trình (workflow)
Strategy	Thay đổi hành vi runtime

🎓 BÀI TẬP MỞ RỘNG (RẤT THỰC CHIẾN)

1. Thêm:
   - `MomoProcessor`
   - `CryptoPaymentProcessor`
2. Thêm bước:
   - `SendEmail()` sau khi thanh toán
3. Log ra DB thay vì Console
4. Kết hợp:
   👉 Template Method + Factory